

1. Information Systems and Software (सूचना प्रणाली और सॉफ्टवेयर)

Definition:

एक Information System (IS) का काम डेटा को इकट्ठा करना, प्रोसेस करना और जानकारी (Information) में बदलना है ताकि सही निर्णय लिए जा सकें। इसमें लोग, हार्डवेयर, सॉफ्टवेयर और डेटा मिलकर काम करते हैं।

- **Key Components:**

1. **Hardware:** कंप्यूटर, सर्वर और डिवाइस।
2. **Software:** प्रोग्राम्स (जैसे OS या Apps) जो डेटा प्रोसेस करते हैं।
3. **Data:** कच्ची जानकारी (Raw facts)।
4. **Procedures:** नियम और प्रोसेस।

- **Software का रोल:** सॉफ्टवेयर वह 'दिमाग' है जो सिस्टम को बताता है कि डेटा के साथ क्या करना है। बिना अच्छे सॉफ्टवेयर के, हार्डवेयर सिर्फ एक डब्बा है।

2. Information Gathering (सूचना एकत्रित करना)

यह **System Development Life Cycle (SDLC)** का शुरुआती चरण है। यहाँ एनालिस्ट यह समझने की कोशिश करता है कि पुराना सिस्टम कैसे काम कर रहा है और नए सिस्टम में क्या चाहिए।

सूचना इकट्ठा करने के मुख्य तरीके (Methods):

- **Interviews:** यूजर्स और मैनेजर से आमने-सामने बात करना।
- **Questionnaires:** बहुत सारे लोगों से फीडबैक लेने के लिए फॉर्म भरवाना।
- **Observation:** खुद जाकर देखना कि असल में काम कैसे हो रहा है।
- **Document Analysis:** पुराने रिकॉर्ड्स, रिपोर्ट्स और फाइलों को पढ़ना।

Note: इसका मुख्य उद्देश्य "Problem Definition" (समस्या को पहचानना) है।

3. Requirement and Feasibility Analysis (आवश्यकता और व्यवहार्यता विश्लेषण)

जब जानकारी इकट्ठा हो जाती है, तब उसे गहराई से जांचा जाता है।

A. Requirement Analysis (आवश्यकता विश्लेषण)

इसमें यह तय किया जाता है कि सिस्टम को **क्या** करना चाहिए।

- **Functional Requirements:** सिस्टम क्या काम करेगा? (जैसे: 'रिपोर्ट जनरेट करना', 'लॉगिन करना')।
- **Non-functional Requirements:** सिस्टम कैसा चलेगा? (जैसे: 'स्पीड', 'सिक्योरिटी', 'यूजर फ्रेंडली होना')।

B. Feasibility Analysis (व्यवहार्यता विश्लेषण)

यहाँ हम यह देखते हैं कि क्या प्रोजेक्ट बनाना संभव है या नहीं? इसे **TELOS** फ्रेमवर्क से समझा जा सकता है:

1. **Technical Feasibility:** क्या हमारे पास सही तकनीक और एक्सपर्ट्स हैं?
2. **Economic Feasibility (Cost-Benefit):** क्या प्रोजेक्ट बजट में है? क्या इसे बनाने से फायदा (ROI) होगा?
3. **Legal Feasibility:** क्या यह कानून और नियमों के खिलाफ तो नहीं है?
4. **Operational Feasibility:** क्या बनने के बाद कर्मचारी या यूजर्स इसे आसानी से इस्तेमाल कर पाएंगे?
5. **Schedule Feasibility:** क्या यह समय सीमा (Deadline) के अंदर पूरा हो सकता है?

Quick Revision Table

टॉपिक	मुख्य सवाल (Key Question)
Information System	हम डेटा कैसे मैनेज कर रहे हैं?
Information Gathering	समस्या क्या है और यूजर को क्या चाहिए?
Requirement Analysis	सिस्टम के फीचर्स क्या-क्या होंगे?
Feasibility Analysis	क्या यह प्रोजेक्ट करना व्यावहारिक है?

4. Data Flow Diagram (DFD - डेटा फ्लो डायग्राम)

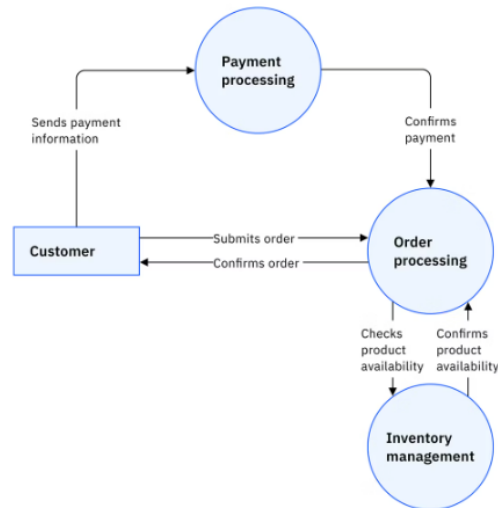
Definition: DFD यह दिखाता है कि एक सिस्टम के भीतर डेटा (Data) कहाँ से आता है, कैसे मूव करता है, कहाँ स्टोर होता है और कहाँ बाहर जाता है। यह सिस्टम के लॉजिक को समझने का एक ग्राफिकल तरीका है।

External Entity		
Process		
Data Store		
Data Flow		

DFD के मुख्य सिम्बल्स (Symbols):

- External Entity (Square/Rectangle):** जहाँ से डेटा आता है या जहाँ जाता है (जैसे: User, Customer)।
- Process (Circle/Rounded Rectangle):** वह काम जो डेटा पर हो रहा है (जैसे: Calculation, Validation)।
- Data Store (Open Rectangle/Parallel Lines):** जहाँ डेटा सेव होता है (जैसे: Database, File)।
- Data Flow (Arrow):** यह डेटा की दिशा बताता है।

Example



Levels of DFD:

- **Level 0 (Context Diagram):** पूरे सिस्टम को एक सिंगल प्रोसेस की तरह दिखाता है।
- **Level 1:** सिस्टम के मुख्य फंक्शन्स को विस्तार में दिखाता है।

5. Process Specification (प्रक्रिया विशिष्टता)

Definition: जब हम DFD बनाते हैं, तो उसमें मौजूद हर "Process" (गोला या बॉक्स) के अंदर असल में क्या काम हो रहा है, उसे विस्तार से लिखना **Process Specification (PSPEC)** कहलाता है।

इसे लिखने के तरीके:

1. **Decision Trees:** एक पेड़ की तरह डायग्राम बनाकर फैसलों को समझना।
2. **Decision Tables:** एक टेबल जिसमें अलग-अलग कंडीशंस और उनके रिजल्ट्स लिखे होते हैं।
3. **Structured English:** कोडिंग जैसी भाषा लेकिन समझने में आसान (जैसे If-Then-Else)।

फायदा: इससे प्रोग्रामर को समझ आ जाता है कि उसे कोड क्या लिखना है।

6. Input/Output Design (इनपुट/आउटपुट डिज़ाइन)

यह यूजर इंटरफेस (UI) का हिस्सा है। इसका मकसद यह सुनिश्चित करना है कि सिस्टम के साथ यूजर का अनुभव आसान हो।

A. Input Design

इसका लक्ष्य डेटा को सिस्टम में बिना किसी गलती के एंटर कराना है।

- **GUI Components:** टेक्स्ट बॉक्स, रेडियो बटन, ड्रॉपडाउन लिस्ट का इस्तेमाल।
- **Validation:** यह चेक करना कि यूजर सही डेटा डाल रहा है (जैसे मोबाइल नंबर में सिर्फ नंबर ही हों)।
- **Simplicity:** फॉर्म भरने में ज्यादा समय न लगे।

B. Output Design

सिस्टम द्वारा प्रोसेस किया गया डेटा यूजर को किस तरह दिखेगा, यह यहाँ तय होता है।

- **Types:** स्क्रीन डिस्प्ले (Soft copy), प्रिंटेड रिपोर्ट (Hard copy), या ईमेल/नोटिफिकेशन।
- **Clarity:** आउटपुट आसान होना चाहिए ताकि मैनेजर या यूजर उसे पढ़कर समझ सकें।
- **Security:** कॉन्फिडेंशियल डेटा (जैसे सैलरी या पासवर्ड) सिर्फ सही व्यक्ति को ही दिखना चाहिए।

Quick Summary for Revision

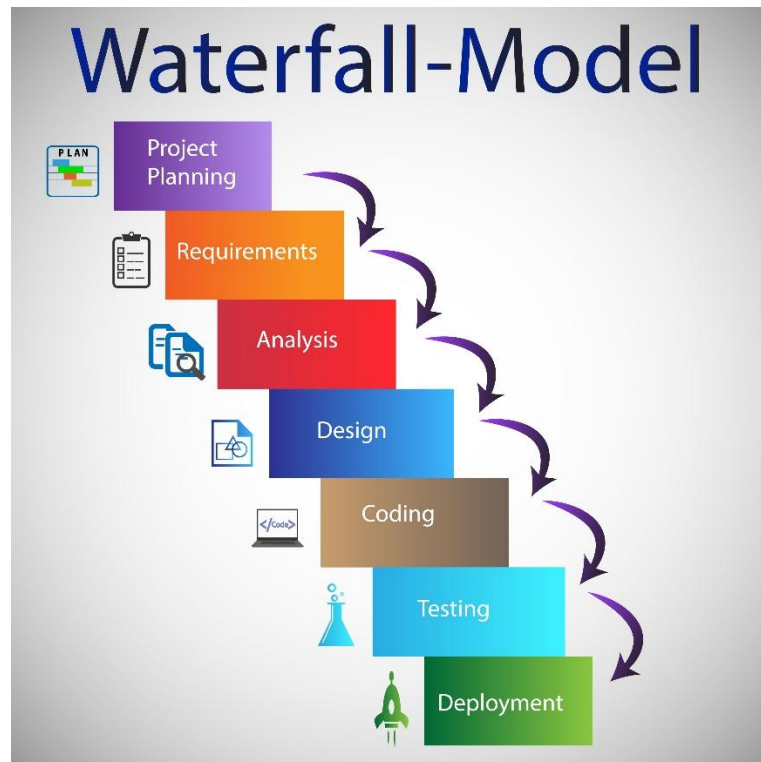
- **DFD:** डेटा के रास्ते का नक्शा (Graph)।
- **Process Spec:** डीएफडी के हर काम की डिटेल (Logic)।
- **I/O Design:** फॉर्म कैसा दिखेगा और रिपोर्ट कैसी बनेगी (Interface)।

7. Process Life Cycle (प्रोसेस लाइफ साइकिल)

इसे अक्सर **SDLC (System Development Life Cycle)** भी कहा जाता है। यह एक ऐसा फ्रेमवर्क है जिसका पालन हर सॉफ्टवेयर को बनाने के लिए किया जाता है।

इसके मुख्य चरण (Stages) इस प्रकार हैं:

1. **Planning (योजना):** प्रोजेक्ट का लक्ष्य तय करना।
2. **Analysis (विश्लेषण):** यूजर की जरूरतों को समझना (Requirement Gathering)।
3. **Design (डिज़ाइन):** सिस्टम का ब्लूप्रिंट तैयार करना (DFD, I/O Design)।
4. **Implementation/Coding (कार्यान्वयन):** प्रोग्रामर्स द्वारा एक्ज्युटल कोड लिखना।
5. **Testing (परीक्षण):** यह चेक करना कि सॉफ्टवेयर में कोई बग (Error) तो नहीं है।
6. **Deployment & Maintenance (रखरखाव):** सॉफ्टवेयर को यूजर को सौंपना और समय-समय पर अपडेट करना।



प्रमुख मॉडल्स: Waterfall Model (सरल), Spiral Model (रिस्क के लिए), और Agile Model (तेजी से काम करने के लिए)।

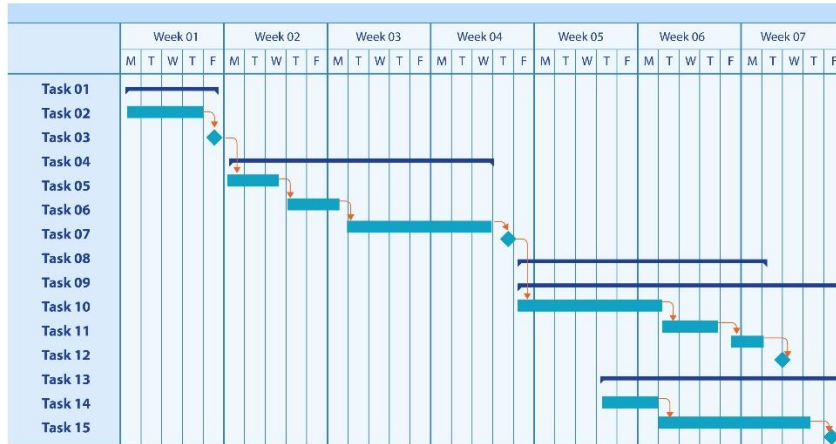
8. Planning and Managing Project (प्रोजेक्ट प्लानिंग और मैनेजमेंट)

सॉफ्टवेयर बनाना सिर्फ कोडिंग नहीं है, इसे मैनेज करना भी एक कला है। इसके दो मुख्य पहलू हैं:

A. Project Planning (नियोजन)

प्रोजेक्ट शुरू होने से पहले ही सब कुछ तय करना।

- **Estimation:** कितना पैसा (Cost) लगेगा? कितने लोग (Effort) चाहिए? और कितना समय (Time) लगेगा?
- **Scheduling:** कौन सा काम कब शुरू होगा और कब खत्म होगा? इसके लिए **Gantt Charts** या **PERT/CPM** का उपयोग किया जाता है।
- **Resource Allocation:** टीम के सदस्यों को उनकी स्किल्स के हिसाब से काम बांटना।



B. Project Managing (प्रबंधन)

प्रोजेक्ट चलते समय उस पर नियंत्रण रखना।

- **Risk Management:** भविष्य में आने वाली समस्याओं (जैसे टीम मेंबर का छोड़ देना या बजट कम पड़ना) का पहले से अंदाजा लगाना।
- **Progress Tracking:** यह देखना कि काम टाइमलाइन के हिसाब से चल रहा है या नहीं।
- **Quality Management:** यह सुनिश्चित करना कि सॉफ्टवेयर की क्वालिटी बेस्ट हो।
- **Configuration Management:** कोड के अलग-अलग वर्जन्स (Versions) को संभाल कर रखना।

Student Tip: Difference Between Planning & Management

विशेषता	Planning (योजना)	Managing (प्रबंधन)
समय	काम शुरू होने से पहले।	काम के दौरान (Execution)।
उद्देश्य	रोडमैप तैयार करना।	रोडमैप पर चलते रहना और रुकावटें हटाना।
टूल्स	WBS (Work Breakdown Structure)	Monitoring Tools, Meetings